Exact algebraic computation

Decomposing Algebraic Roots into Low-Degree Sums and Products

Mathematica algorithms, optimality certificates, and the role of the splitting field

Technical study prepared for Vladimir Reshetnikov

6 September 2026

Abstract

An algebraic number of degree nine can be a sum or a product of two cubic numbers, but finding such an identity and proving that it minimizes the largest component degree are different tasks. We give exact solutions and optimality proofs for both examples in Mathematica StackExchange question 105933. We then develop a terminating mathematical algorithm for the unrestricted additive problem, using fixed fields in a Galois closure, and a terminating algorithm for the unrestricted *two-factor* product problem, using relative norms and rational linear algebra.

A concrete degree-eight example shows that optimal multiplicative factors can lie outside even the input's splitting field. It also disproves a published multiplicative descent assertion whose proof divides by a possibly zero coefficient. The corrected two-factor criterion explicitly retains a bounded radical exponent. For arbitrarily many factors we supply exact, certificate-guided bounded search, not a claimed general terminating minimizer. The accompanying Wolfram Language package implements these procedures; independent exact Python checks verify the polynomial identities, minimal degrees, and finite-group calculations.

Scope and validation. Both requested degree-nine examples are settled globally: the optimal maximum degree is **three**, even with arbitrarily many components allowed. General sums and general two-factor products have complete finite algorithms here. The arbitrary-length product problem is covered by certified search and lower bounds. Native Wolfram execution could not be completed because the available evaluator returned HTTP 404; the supplied native test suites are therefore unexecuted.

Contents

1 The problem, made precise

For an exact algebraic number a, write

$$\deg_{\mathbb{Q}}(a) = [\mathbb{Q}(a) : \mathbb{Q}].$$

The degree is always the *absolute degree over the rationals*, not the degree of an arbitrary polynomial that happens to vanish at a, and not a relative extension degree over a field already containing other components. Define

$$D_{+}(a) = \min\left\{\max_i \deg_{\mathbb{Q}}(b_i) : a = b_1 + \cdots + b_r, \quad r \geq 1\right\}, \tag{1}$$

$$D_{\times}(a) = \min\left\{\max_i \deg_{\mathbb{Q}}(b_i) : a = b_1 \cdots b_r, \quad r \geq 1,\ b_i \neq 0\right\}, \tag{2}$$

where the b_i may be anywhere in $\overline{\mathbb{Q}}$. The multiplicative definition initially assumes $a \neq 0$; zero has the trivial degree-one representation. Rationals have degree one. Permitting a single component makes the feasible upper bound $\deg_{\mathbb{Q}}(a)$ explicit. Requiring at least two components instead changes nothing, because zero or one can be appended.

The number of components is not the objective. Nor are coefficient height, radical nesting, expression length, or numerical conditioning. Those are useful secondary objectives, but optimizing them must not silently replace (1) or (2).

We will also need the genuinely different quantity

$$D_{\times,2}(a) = \min_{a=bc} \max\{\deg_{\mathbb{Q}}(b), \deg_{\mathbb{Q}}(c)\}.$$

Always $D_{\times}(a) \leq D_{\times,2}(a)$, and strict inequality occurs. A complete algorithm for two factors is consequently not, by itself, a complete algorithm for the question as stated.

Three ambient domains must also be distinguished:

$$K = \mathbb{Q}(a) \ \subseteq\ L = \text{the splitting field of the minimal polynomial of } a \ \subseteq\ \overline{\mathbb{Q}}.$$

Restricting components to K can change the answer. For sums it is enough to work in L, by a trace argument. For products even restriction to L can change the answer. These distinctions drive the implementation.

2 The two examples: exact answers and short certificates

Let u and v be the unique real roots of

$$p(x) = x^3 + x + 1, \qquad q(x) = x^3 - x + 1.$$

Both cubics are irreducible over $\mathbb{Q}$ by the rational root test. Their discriminants are -31 and -23, respectively, so each has one real root. Put $a = uv$ and $s = u + v$. Their minimal polynomials are

$$P_{\times}(x) = x^9 + 2x^7 - 3x^6 + x^5 - x^4 + 3x^3 - x - 1, \tag{3}$$

$$P_{+}(x) = x^9 + 6x^6 + 3x^5 - 15x^3 + 24x^2 - 4x + 8. \tag{4}$$

The exact verification script supplied with this article checks irreducibility and counts the real roots of both polynomials; each has exactly one. Thus the first real `Root` in each example is the intended number.

2.1 A compact Mathematica answer

```
p = x^3 + x + 1;
q = x^3 - x + 1;
u = Root[1 + # + #^3 &, 1];
v = Root[1 - # + #^3 &, 1];

ap = Root[-1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 +
          2 #^7 + #^9 &, 1];
as = Root[8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 +
          6 #^6 + #^9 &, 1];

RootReduce /@ {ap - u v, as - u - v}
(* Expected: {0, 0} *)

Exponent[MinimalPolynomial[#, x], x] & /@ {ap, as, u, v}
(* Expected: {9, 9, 3, 3} *)
```

`RootReduce` performs exact algebraic normalization; it does not seek this decomposition objective [5]. Keeping the components in a list, or displaying an inactive sum/product, avoids immediately collapsing an answer back into a single algebraic number.

The expected display values, not used as proof, are

$$u \approx -0.68232780382801932737, \quad v \approx -1.32471795724474602596,$$

$$a \approx 0.90389189445834756110, \quad s \approx -2.00704576107276535333.$$

2.2 Recovering the cubics directly from the degree-nine inputs

The following rational functions are particularly short exact certificates:

$$u = \frac{a^3 - a^2 - 1}{a^4 + a^2 - a + 1}, \qquad v = \frac{a}{u}, \tag{5}$$

and

$$u = \frac{3s^5 - 5s^3 - 3s^2 + 2s - 4}{6s^4 - 6s + 4}, \qquad v = s - u. \tag{6}$$

Their denominators are relatively prime to the respective input minimal polynomials, so they do not vanish at any conjugate of the input. Substituting these expressions into $p(u)$ and $q(v)$ gives zero modulo $P_\times(a)$ or $P_+(s)$. Since all values in these formulas are real, the unique real roots of the cubics are selected automatically.

For example, the product calculation can be reproduced entirely with polynomial remainders:

```
P = x^9 + 2 x^7 - 3 x^6 + x^5 - x^4 + 3 x^3 - x - 1;
num = x^3 - x^2 - 1;
den = x^4 + x^2 - x + 1;
{
 PolynomialGCD[P, den],
 PolynomialRemainder[num^3 + num den^2 + den^3, P, x],
 PolynomialGCD[P, num],
 PolynomialRemainder[(x den)^3 - (x den) num^2 + num^3, P, x]
}
(* Expected: {1, 0, 1, 0} *)
```

The first remainder certifies $p(\mathrm{num}/\mathrm{den}) = 0$; the second certifies $q(x\,\mathrm{den}/\mathrm{num}) = 0$. No numerical root matching is involved. Rational recovery formulas can be obtained by a polynomial GCD or extended

Euclidean computation over $\mathbb{Q}(a)$ after eliminating one variable. They are compact certificates, not a general discovery algorithm.

An alternative product certificate involves no denominators depending on a:

$$u = -\frac{a^7 + 2a^5 - 2a^4 + 1}{2},$$
$$v = -\frac{a^7 + 2a^5 - 2a^4 + 2a^3 + 2a - 1}{2}.$$

It also demonstrates explicitly that both cubic factors lie in $\mathbb{Q}(a)$ for this particular example. That fortunate property is not a general theorem.

2.3 Why degree three is optimal, not merely successful

Suppose a number is a sum or product of finitely many numbers of degree at most two. Each component lies in a quadratic or rational field. Their compositum is a multiquadratic extension and has degree a power of two. Every element of that compositum has degree dividing that power of two. A degree-nine number cannot occur. Consequently both displayed representations attain the global minimum:

$$\boxed{D_{\times}(a) = 3, \qquad D_{+}(s) = 3.}$$

This rules out *any number* of quadratic components, even in auxiliary fields not suggested by the original expression. Section 4 generalizes this certificate.

3 Discovering candidates with resultants and exact residuals

3.1 Forward construction is easy

If $p(u) = q(v) = 0$, then an annihilating polynomial for $u + v$ is

$$R_{+}(x) = \operatorname{Res}_y\big(p(y), q(x - y)\big).$$

If $m = \deg q$, an annihilating polynomial for uv is

$$R_{\times}(x) = \operatorname{Res}_y\big(p(y), y^m q(x/y)\big).$$

The second argument in the latter expression is polynomial after clearing the displayed denominator. The companion package implements these as `SumPolynomial` and `ProductPolynomial`, using the documented `Resultant` operation [9].

For the examples:

```
Resultant[y^3 + y + 1, (x-y)^3 - (x-y) + 1, y]
(* Pplus[x] *)
Resultant[y^3 + y + 1, x^3 - x y^2 + y^3, y]
(* Ptimes[x] *)
```

A characteristic polynomial of a Kronecker sum or Kronecker product of companion matrices gives the same forward information. The existing StackExchange answers use these sorts of constructions and coefficient matching [1]. They are useful discovery devices, but do not by themselves prove minimality.

3.2 Why blindly inverting a resultant is insufficient

For unknown monic polynomials p and q, one can equate resultant coefficients to those of a target polynomial and solve the resulting system. This is useful at very small degrees, especially after a plausible shape has been identified. Several qualifications are essential.

First, the input minimal polynomial only has to *divide* the resultant. Equality is not generally necessary: repeated conjugate products or sums can make the resultant a power of a smaller polynomial. In Section 7, two quartics have a degree-eight product and the resultant is the square of its minimal polynomial.

Second, unknown coefficients must be rational. A complex or algebraic solution of the coefficient equations does not give the intended absolute degree bound. Third, integer monic polynomials only cover algebraic integers. A complete finite-height dictionary must include nonmonic primitive integer minimal polynomials. Finally, convenient normalizations such as constant term one or simultaneously depressed cubics need their own justification; rational rescaling cannot always achieve them.

After any candidate polynomials have been found, enumerate their exact root branches and verify the actual input value. A resultant identifies possible conjugate values, not a particular branch. Mathematica places real polynomial roots first in increasing order, but complex-root ordering has additional method-dependent details [4]. Do not reuse a numerically guessed root index as a universal branch rule.

3.3 The residual trick: search for one polynomial, not two

When a candidate polynomial p is available, enumerate its roots b and compute

$$c = a - b \quad \text{or} \quad c = a/b.$$

Then calculate the exact minimal polynomial of c. The second unknown polynomial does not need to be searched independently. This reduces a candidate-pair problem to a candidate-singleton problem.

After loading the package, the examples are found by

```
Get["code/RootDecomposition.wl"];
PairFromPolynomial[ap, x^3+x+1, x, "Product", 3]
PairFromPolynomial[as, x^3+x+1, x, "Sum", 3]
```

A successful association contains the component list, each absolute degree, the maximum degree, exact equality status, and an optimality flag. A failed trial means only that *this candidate polynomial* did not produce a pair under the degree cap.

Without advance knowledge of either cubic, the following bounded calls search the primitive integer polynomials of degree at most three and coefficient height at most one:

```
SearchRootDecomposition[ap, "Product", 3, 1, 2]
SearchRootDecomposition[as, "Sum",     3, 1, 2]
```

Both cubics belong to that dictionary. The exact arithmetic checks and the prime-divisor bound provide the certificates; dictionary membership is only the discovery mechanism. The search implementation and its precise completeness statement are discussed in Section 9.

4 Lower bounds that survive arbitrary auxiliary fields

Theorem 4.1 (Normal-closure obstruction)**.** *Let a have degree n, let L be its splitting field, and put $G = \mathrm{Gal}(L/\mathbb{Q})$. If a is a finite sum or product of algebraic numbers of degree at most d, then*

$$\textit{every prime divisor of } |G| \textit{ is at most } d, \qquad \exp(G) \mid \mathrm{lcm}(1, 2, \ldots, d). \tag{7}$$

In particular, the largest prime divisor of n is a lower bound for both $D_+(a)$ and $D_\times(a)$.

Proof. For every component b_i, let M_i be the normal closure of $\mathbb{Q}(b_i)$. If $m_i = \deg_{\mathbb{Q}}(b_i)$, the action on its m_i conjugates embeds $\mathrm{Gal}(M_i/\mathbb{Q})$ in S_{m_i}. Let M be the compositum of these normal closures. Restriction embeds $\mathrm{Gal}(M/\mathbb{Q})$ into their direct product. Since $M/\mathbb{Q}$ is normal and contains a, it contains every conjugate of a, hence L. Restriction onto G is surjective.

All prime divisors of the order of a subgroup of the direct product are at most d. Its exponent divides $\mathrm{lcm}(1, \ldots, d)$, because that is a common multiple of the orders of all permutations in each S_{m_i}. These properties pass to a quotient. Finally n divides $[L : \mathbb{Q}] = |G|$. □

Corollary 4.2. *A number of prime degree p cannot be expressed as a finite sum or product of numbers all having degrees less than p.*

The inexpensive implementation is

```
PrimeDegreeBound[a]
```

which needs only `MinimalPolynomial` and integer factorization. Once Galois data are available, the package strengthens it using the least d for which the exponent divisibility in (7) holds. For example, exponent four rules out degree caps two *and three*.

A separate, length-dependent bound is

$$\deg_{\mathbb{Q}}(a) \le \prod_{i=1}^{r} \deg_{\mathbb{Q}}(b_i) \le d^r. \tag{8}$$

It follows from the usual degree inequality for a compositum. For two components it gives $d \ge \lceil \sqrt{n} \rceil$.

Remark 4.3. Do not strengthen the compositum inequality to an unjustified divisibility assertion. For nonnormal fields, the compositum degree need not divide the product of their degrees: two distinct conjugate cubic fields in an S_3 splitting field can have compositum degree six, although their degree product is nine. The prime-divisor proof above deliberately uses normal closures, whose groups embed in symmetric groups.

These bounds are necessary, not generally sufficient. A certificate saying `GlobalOptimal -> False` in the lightweight checker means that global optimality was *not established*, not that the representation is known to be suboptimal.

5 A complete solution for arbitrary finite sums

5.1 Why all summands can be moved into a finite field

The following is the decisive additive descent principle. An additive normal-closure reduction also appears in Altamirano's study [2]; we give the argument explicitly, including its componentwise degree bound.

Lemma 5.1 (Trace projection)**.** *Let $L/\mathbb{Q}$ be finite Galois, let $a \in L$, and suppose $a = \sum_i b_i$ with arbitrary algebraic b_i. Then there are $c_i \in L \cap \mathbb{Q}(b_i)$ with*

$$a = \sum_i c_i, \qquad \deg_{\mathbb{Q}}(c_i) \le \deg_{\mathbb{Q}}(b_i).$$

Proof. Choose a finite Galois extension $M/\mathbb{Q}$ containing L and every b_i, and put $N = \mathrm{Gal}(M/L)$. Define

$$\pi_L(x) = \frac{1}{|N|} \sum_{\sigma \in N} \sigma(x).$$

This is a $\mathbb{Q}$-linear projection onto L, so $a = \pi_L(a) = \sum_i \pi_L(b_i)$. It remains to control degree, rather than merely field membership.

Because $L/\mathbb{Q}$ is Galois, N is normal in $\mathrm{Gal}(M/\mathbb{Q})$. If h fixes b_i, conjugation by h permutes N, and therefore $h\pi_L(b_i) = \pi_L(hb_i) = \pi_L(b_i)$. Thus $\pi_L(b_i)$ is fixed by $\mathrm{Gal}(M/\mathbb{Q}(b_i))$ as well as by N. It lies in $\mathbb{Q}(b_i) \cap L$. Set $c_i = \pi_L(b_i)$. □

The normality assumption is essential to the equivariance step. Averaging onto the generally nonnormal field $\mathbb{Q}(a)$ need not preserve the component degree. The correct finite ambient field is the splitting field L.

5.2 Replace unknown algebraic numbers by fixed spaces

Let $G = \mathrm{Gal}(L/\mathbb{Q})$. For $H \leq G$, the fixed field L^H has absolute degree $[G : H]$. Define the rational vector subspace

$$W_d = \sum_{\substack{H \leq G \\ [G:H] \leq d}} L^H \subseteq L. \tag{9}$$

The sum here is a *linear sum of subspaces*. It is not the compositum of those fields. Passing to the compositum would allow products and would solve a different problem.

Theorem 5.2 (Exact additive criterion)**.** *For an algebraic number a with splitting field L,*

$$D_+(a) \leq d \quad \Longleftrightarrow \quad a \in W_d.$$

Consequently $D_+(a)$ is computable by finite subgroup enumeration and rational linear algebra.

Proof. If a representation with maximum degree at most d exists, apply Lemma 5.1. Each projected summand c_i lies in a field $\mathbb{Q}(c_i) \subseteq L$ of degree at most d, so it lies in one of the summands in (9). Conversely, a vector-space representation in W_d is a finite sum of elements of the listed fields, each of degree at most d.

There are finitely many subgroups of the finite group G. Test d in increasing order up to $n = \deg_{\mathbb{Q}}(a)$. The test must succeed by $d = n$, because $a \in \mathbb{Q}(a) \subseteq L$. □

A rational scalar multiplying a basis vector can be absorbed into that summand without increasing degree. Selecting an independent subset of the combined field bases shows that no more than $[L : \mathbb{Q}]$ individual basis contributions are ever needed. In practice, grouping all contributions from the same fixed field often gives a much shorter answer.

5.3 The exact matrix computation

Choose a primitive element θ for L and use the power basis $1, \theta, \ldots, \theta^{m-1}$, where $m = [L : \mathbb{Q}]$. Represent field elements as rational column vectors. For each $g \in G$, let A_g be its $m \times m$ rational matrix. A basis matrix for L^H is obtained from

$$\bigcap_{h \in H} \ker(A_h - I).$$

Stacking the equations and calling `NullSpace` computes this basis. For a fixed degree cap, concatenate the eligible field bases into a matrix B_d. Then

$$a \in W_d \quad \Longleftrightarrow \quad B_d c = \mathrm{vec}(a)$$

has a rational solution. `LinearSolve` supplies one when it exists [10]. Its coefficients are grouped by field and converted back into exact algebraic numbers.

The implementation stores fixed-field bases as *rows*; it transposes them when forming B_d. Automorphism and multiplication matrices act on *columns*. This convention matters: accidentally transposing one operator can yield a plausible-looking but incorrect subfield calculation.

```
g = BuildGaloisData[as];
r = CompleteSumDecomposition[as, g];
r["MaximumDegree"]       (* expected 3 *)
r["GlobalOptimal"]       (* expected True *)
```

The example does not require this expensive route because the cubic candidate and the prime bound already settle it. The fixed-space method is the general completeness mechanism.

5.4 Why the procedure is finite but can be expensive

A degree-n polynomial may have splitting field degree as large as $n!$. All computations above take place in that field, not necessarily in the much smaller field $\mathbb{Q}(a)$. Exact coefficients can also grow considerably. The supplied backend is an explicit small-field reference implementation, not a claim of a polynomial-time simplifier.

The defaults limit the constructed field to degree 128 and subgroup enumeration to 10,000 entries. Reaching either guard produces a `Failure`, not an impossibility certificate. The field-degree guard is checked *after* primitive-element construction; it does not bound the time spent constructing that element. An outer `TimeConstrained` can impose an operational time limit, again with an inconclusive result.

6 The product of two factors: a complete norm–intersection algorithm

Unlike additive projection, a field norm preserves a product but raises the input to a power. That power must be retained. It is precisely what permits optimal factors outside the splitting field.

6.1 An intersection lemma for Galois base fields

Lemma 6.1. *Let $L/\mathbb{Q}$ be finite Galois and b algebraic. Put $E = \mathbb{Q}(b)$, $E_0 = E \cap L$, and $t = [L(b) : L]$. Then*

$$[E : \mathbb{Q}] = t[E_0 : \mathbb{Q}], \qquad \mathrm{N}_{L(b)/L}(b) \in E_0.$$

The minimal polynomial of b over L is the same polynomial as its minimal polynomial over E_0.

Proof. The compositum LE/E is Galois. Restriction identifies its Galois group with a subgroup of $\mathrm{Gal}(L/\mathbb{Q})$ whose fixed field in L is $L \cap E$. Thus $[LE : E] = [L : E_0]$. The tower formula gives $[LE : L] = [E : E_0]$. The minimal polynomial over E_0 has this latter degree, remains a polynomial over L, and vanishes at b, so it is already minimal over L. Its constant coefficient, up to sign, is the displayed norm, and belongs to E_0. □

This standard norm/intersection fact is also stated in Samuels's Lemma 2.2 [3]. Its relevance here is the exact absolute-degree factorization $t[E_0 : \mathbb{Q}]$, not a height estimate.

6.2 A finite necessary-and-sufficient criterion

Theorem 6.2 (Two-factor criterion)**.** *Let $0 \neq a \in L$, with $L/\mathbb{Q}$ finite Galois, and let $d \geq 1$. There exist $b, c \in \overline{\mathbb{Q}}$ such that*

$$a = bc, \qquad \deg_{\mathbb{Q}}(b), \deg_{\mathbb{Q}}(c) \leq d$$

if and only if there are subfields $E_0, F_0 \subseteq L$ and an integer $t \geq 1$ for which

$$t[E_0 : \mathbb{Q}] \leq d, \qquad t[F_0 : \mathbb{Q}] \leq d, \qquad E_0 \cap a^t F_0 \neq \{0\}. \tag{10}$$

If $n = \deg_{\mathbb{Q}}(a)$, it suffices to consider

$$1 \leq t \leq \min\left(d, \left\lfloor \frac{d^2}{n} \right\rfloor\right). \tag{11}$$

Proof. For necessity, suppose $a = bc$. Then $L(b) = L(c) = M$, because $c = a/b$ and $b = a/c$. Put $t = [M : L]$, $E_0 = L \cap \mathbb{Q}(b)$, and $F_0 = L \cap \mathbb{Q}(c)$. By Lemma 6.1,

$$\deg_{\mathbb{Q}}(b) = t[E_0 : \mathbb{Q}], \qquad \deg_{\mathbb{Q}}(c) = t[F_0 : \mathbb{Q}].$$

Let $u = \mathrm{N}_{M/L}(b)$ and $v = \mathrm{N}_{M/L}(c)$. These nonzero values belong to E_0 and F_0, respectively, and

$$uv = \mathrm{N}_{M/L}(a) = a^t.$$

Therefore $u = a^t/v \in E_0 \cap a^t F_0$, establishing (10).

For sufficiency, choose any nonzero $u \in E_0 \cap a^t F_0$. Choose a t-th root b of u, and define $c = a/b$. Then

$$b^t = u \in E_0, \qquad c^t = a^t/u \in F_0.$$

The polynomials $X^t - u$ over E_0 and $X^t - a^t/u$ over F_0 give

$$\deg_{\mathbb{Q}}(b) \leq t[E_0 : \mathbb{Q}] \leq d, \qquad \deg_{\mathbb{Q}}(c) \leq t[F_0 : \mathbb{Q}] \leq d.$$

There is no root-of-unity ambiguity in the product: the second factor was defined by division, rather than by an independent root choice.

Finally, writing $e = [E_0 : \mathbb{Q}]$ and $f = [F_0 : \mathbb{Q}]$, the condition implies $a^t \in E_0 F_0$. Consequently

$$n \leq t \deg_{\mathbb{Q}}(a^t) \leq t[E_0 F_0 : \mathbb{Q}] \leq tef \leq d^2/t.$$

Thus $t \leq d^2/n$, and $t \leq d$ follows already from $te \leq d$. □

The same proof works with unequal degree caps d_1, d_2, replacing the first two inequalities by $te \leq d_1$, $tf \leq d_2$, and the upper bound by $t \leq \min(d_1, d_2, \lfloor d_1 d_2/n \rfloor)$. The companion package uses the equal-cap form because that is the requested objective.

6.3 Everything before the final roots is rational linear algebra

Let B_E, B_F be column basis matrices of E_0, F_0 in a common power basis of L. Let M_{a^t} be the rational matrix of multiplication by a^t. A nonzero element in the intersection is obtained from a nonzero solution of

$$\begin{bmatrix} B_E & -M_{a^t} B_F \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0. \tag{12}$$

Because both field bases are linearly independent and $a \neq 0$, a nonzero null vector cannot give $B_E x = 0$: that would also force $y = 0$. Thus a single null-space basis vector suffices. Set $u = B_E x$, take one exact t-th root, and obtain the other factor by division.

In the special case $t = 1$, this is the elementary but powerful criterion

$$a \in E_0^{\times} F_0^{\times} \quad \Longleftrightarrow \quad E_0 \cap aF_0 \neq \{0\}.$$

No factorization of ideals or computation of unit groups is needed for this *two-fixed-field* membership problem. The unknown factors are unrestricted field elements, not units or integral elements.

The resulting finite algorithm is to enumerate d increasingly, then the bounded t in (11), then all eligible embedded subfields, and test (12). It starts at

$$\max\{\lceil\sqrt{n}\rceil,\ \text{available global lower bounds}\}$$

and stops no later than $d = n$, where the pair $a \cdot 1$ is feasible.

```
g = BuildGaloisData[ap];
r = CompleteTwoFactorDecomposition[ap, g];
r["MaximumDegree"]          (* expected 3 *)
r["OptimalAmongTwoFactors"] (* expected True *)
r["GlobalOptimal"]          (* expected True: lower bound also 3 *)
```

The first optimality statement comes from exhaustive testing of the finite criterion. The second is stronger and is set only when a lower bound valid for arbitrarily many factors matches the returned degree. These flags must not be conflated.

7 A counterexample: optimal factors outside the splitting field

Consider the positive real number

$$a = \sqrt{(1+\sqrt{2})(1+\sqrt{3})}. \tag{13}$$

It has the evident factorization

$$a = \underbrace{\sqrt{1+\sqrt{2}}}_{b}\ \underbrace{\sqrt{1+\sqrt{3}}}_{c}. \tag{14}$$

The factors have irreducible quartic minimal polynomials

$$X^4 - 2X^2 - 1, \qquad X^4 - 2X^2 - 2.$$

The minimal polynomial of a is the irreducible degree-eight polynomial

$$P(X) = X^8 - 4X^6 - 16X^4 - 8X^2 + 4. \tag{15}$$

Here a is the *fourth* real root in increasing order, not the first. Using the positive radical expression avoids any index guess.

We will prove substantially more than failure of the particular factors to lie in the splitting field: *no product of any number of elements of that splitting field, all of degree less than eight, can equal a*. Nevertheless (14) attains the unrestricted optimum four.

7.1 The splitting field has degree sixteen

Write $r = \sqrt{2}$, $s = \sqrt{3}$, $A = 1 + r$, and $B = 1 + s$. Set

$$B_0 = \mathbb{Q}(r, s), \qquad B_1 = B_0(i), \qquad L = B_1(a).$$

The four conjugates of $a^2 = AB$ are distinct, so $\mathbb{Q}(a^2) = B_0$. At some real embeddings of B_0, the value of AB is negative; it therefore cannot be a square in the totally real field B_0. It follows that $[\mathbb{Q}(a) : \mathbb{Q}] = 8$.

The real elements of $B_1 = B_0(i)$ are precisely B_0. Since a is real but not in B_0, it is not in B_1. Thus $[L : \mathbb{Q}] = 16$. All the conjugates of a lie in L and are

$$\pm a, \qquad \pm iB/a, \qquad \pm irA/a, \qquad \pm r/a. \tag{16}$$

Conversely, the splitting field contains $\mathbb{Q}(a^2) = B_0$ and the product $a(iB/a) = iB$, hence also i. Therefore L is exactly the splitting field, rather than merely a convenient larger normal extension.

7.2 The automorphisms force all small subfields into one place

Let z fix B_1 and send a to $-a$. Define involutions σ, τ, and κ by

	r	s	i	a
σ	$-r$	s	i	iB/a
τ	r	$-s$	i	irA/a
κ	r	s	$-i$	a

where κ is complex conjugation. Direct substitution into the field relations gives

$$z^2 = \sigma^2 = \tau^2 = \kappa^2 = 1, \qquad z \text{ central},$$

$$[\sigma, \tau] = [\sigma, \kappa] = [\tau, \kappa] = z.$$

Every group element has a unique form $\sigma^e \tau^f \kappa^g z^h$, with exponents in $\{0, 1\}$, and

$$(\sigma^e \tau^f \kappa^g z^h)^2 = z^{ef+eg+fg}. \tag{17}$$

The exponent of $\mathrm{Gal}(L/\mathbb{Q})$ is four.

Lemma 7.1. *Every subgroup of* $\mathrm{Gal}(L/\mathbb{Q})$ *having order at least four contains* z.

Proof. Every element of order four squares to z. Thus a subgroup of order four avoiding z would have to be a Klein four group. By (17), a noncentral involution must lie in one of the three cosets $\sigma\langle z\rangle$, $\tau\langle z\rangle$, or $\kappa\langle z\rangle$. Involutions from different such cosets do not commute; distinct involutions from the same coset multiply to z. Hence no Klein four subgroup avoids z. Every larger subgroup, being a finite 2-group, contains a subgroup of order four. □

Every subfield of L with degree less than eight has degree one, two, or four. Its corresponding subgroup has order at least four, so the subfield is contained in

$$L^{\langle z\rangle} = B_1.$$

Since $a \notin B_1$, no sum or product of elements of these smaller subfields can equal a. This argument allows arbitrarily many such elements.

7.3 Exact unrestricted optima

The quartic factorization gives $D_\times(a) \le 4$. The group exponent four and Theorem 4.1 exclude degree caps at most three. Hence

$$\boxed{D_\times(a) = D_{\times,2}(a) = 4.}$$

For sums, Lemma 5.1 would move any lower-degree decomposition into L without increasing the summand degrees. The preceding fixed-field obstruction rules this out, giving

$$\boxed{D_+(a) = 8.}$$

Thus the same degree-eight number has an optimal quartic product but no lower-degree additive decomposition at all.

The two-factor theorem finds (14) through

$$t = 2, \qquad E_0 = \mathbb{Q}(\sqrt{2}), \qquad F_0 = \mathbb{Q}(\sqrt{3}), \qquad u = 1 + \sqrt{2}.$$

Indeed $a^2/u = 1 + \sqrt{3}$. Omitting $t > 1$ would miss this globally optimal answer.

```
aext = RootReduce[Sqrt[(1+Sqrt[2]) (1+Sqrt[3])]];
gext = BuildGaloisData[aext];
CompleteTwoFactorDecomposition[aext, gext]
(* expected maximum degree 4; norm exponent 2; globally optimal *)
CompleteSumDecomposition[aext, gext]
(* expected maximum degree 8; globally optimal *)
```

The actual factors returned by a null-space basis need not be the positive radicals displayed above. Any exact branch is permitted by the original algebraic problem, and the second factor is constructed by division.

7.4 A specific correction to a published descent claim

Altamirano's Theorem 3.1 asserts that two lower-degree multiplicative factors can always be replaced by lower-degree factors inside the Galois closure [2, Theorem 3.1, p. 5]. The example above disproves that assertion. In the printed proof, the proposed replacement uses a_0/a_1, where a_1 is the linear coefficient of a relative minimal polynomial; there is no justification that $a_1 \neq 0$. Here the relevant polynomial is $X^2 - (1 + \sqrt{2})$ and the linear coefficient is zero. Retaining the relative exponent in Theorem 6.2 avoids this failure. This correction concerns that multiplicative descent assertion, not the valid additive trace reduction.

The quartic product also provides a useful regression test for resultant inversion:

```
ProductPolynomial[x^4-2 x^2-1, x^4-2 x^2-2, x, z]
(* expected (z^8-4 z^6-16 z^4-8 z^2+4)^2, expanded *)
```

Requiring the resultant itself to equal the input minimal polynomial would reject this genuine factorization.

8 An extra result for the original product: its best sum uses sextics

The original product $a = uv$ has degree nine and $D_\times(a) = 3$. Its additive optimum is different:

$$\boxed{D_+(uv) = 6.} \tag{18}$$

Moreover, restricting summands to $\mathbb{Q}(uv)$ raises the minimum to nine. This example shows why the additive algorithm needs the whole splitting field rather than just the input field.

8.1 A concrete sextic representation

Let the roots of p be u_1, u_2, u_3, and those of q be v_1, v_2, v_3, with $u_1 = u$ and $v_1 = v$. For $\pi \in S_3$, put

$$w_\pi = \sum_{j=1}^{3} u_j v_{\pi(j)}.$$

There are six such pairing values. The two permutations fixing the real-root pair give

$$w_{\mathrm{id}} + w_{(23)} = 2u_1v_1 + (u_2 + u_3)(v_2 + v_3) = 3u_1v_1.$$

Both terms on the left are real, because the remaining roots occur in complex-conjugate pairs. Their common irreducible polynomial is

$$\begin{aligned} R(X) &= X^6 + 6X^4 - 27X^3 + 9X^2 - 81X + 4 \\ &= \left(X^3 + 3X - \frac{27}{2}\right)^2 - \frac{713}{4}. \end{aligned} \tag{19}$$

Each of the two cubics obtained by choosing a sign of $\sqrt{713}$ has strictly positive derivative, so R has exactly two real roots. Consequently

```
w1 = Root[#^6 + 6 #^4 - 27 #^3 + 9 #^2 - 81 # + 4 &, 1];
w2 = Root[#^6 + 6 #^4 - 27 #^3 + 9 #^2 - 81 # + 4 &, 2];
RootReduce[ap - (w1+w2)/3]
(* expected 0 *)
```

The summands are $w_1/3$ and $w_2/3$, each still of degree six. If literal `Root` terms with no outside scalar are preferred, use the two real roots of $R(3X)$.

There is a short algebraic way to check this resolvent. The two real pairings satisfy

$$w^2 - 3uv\,w + (3u^2 - 3v^2 + 4) = 0. \tag{20}$$

Indeed $(u_2 - u_3)^2 = -3u^2 - 4$ and $(v_2 - v_3)^2 = 4 - 3v^2$. The polynomial certificates in Section 2 express u, v in $\mathbb{Q}(a)$. Substituting them shows that (20) divides $R(w)$ in $\mathbb{Q}(a)[w]$. This exact reduction is checked independently in `verification/verify.py`.

8.2 Why no five-degree summands can suffice

The splitting fields of the two irreducible cubics have groups S_3 and have distinct quadratic subfields $\mathbb{Q}(\sqrt{-31})$ and $\mathbb{Q}(\sqrt{-23})$. Their intersection, being normal over $\mathbb{Q}$, must be trivial: the only proper nontrivial normal subfield of an S_3 splitting field is its quadratic subfield. Their compositum therefore has group

$$G = S_3 \times S_3, \qquad |G| = 36.$$

Let $U = \operatorname{span}_{\mathbb{Q}}\{u_1, u_2, u_3\}$ and $V = \operatorname{span}_{\mathbb{Q}}\{v_1, v_2, v_3\}$. They are the two-dimensional standard representations, because their only rational root-sum relation is zero. The products span the four-dimensional irreducible representation $T = U \otimes V \subset L$, and $a = u_1 v_1$ is a nonzero vector of T. Linearly disjoint splitting fields ensure the tensor multiplication map is injective.

We claim $T^H = 0$ for every subgroup $H \le G$ of index at most five. The possible indices are $1, 2, 3, 4$. Let $A = C_3 \times C_3$ be the normal Sylow 3-subgroup of G. For indices one, two, and four, H contains A, which has no invariant vector in T. For index three, $|H| = 12$, $H \cap A$ has order three, and the image of H in $G/A = C_2 \times C_2$ is the whole quotient. Viewing A as a two-dimensional vector space over $\mathbb{F}_3$, its order-three subgroup $H \cap A$ must be invariant under independent sign changes of the two coordinates. It must therefore be a coordinate axis. Thus H contains $C_3 \times 1$ or $1 \times C_3$, again forcing $T^H = 0$.

Finite-group representations over $\mathbb{Q}$ admit equivariant projections onto subrepresentations: average any linear projection over the group. Project L onto T. Every element of every fixed field of degree at most five projects to zero, whereas a projects to itself. Therefore $a \notin W_5$. The sextic representation proves $a \in W_6$, establishing (18).

Finally, $\mathbb{Q}(a) = \mathbb{Q}(u_1, v_1)$ has degree nine, and its stabilizer is $H_0 = S_2 \times S_2$, of order four. A subfield of $\mathbb{Q}(a)$ corresponds to a subgroup containing H_0. Its order must be a multiple of four dividing 36, so the only field degrees are nine, three, and one. All proper subfields have degree at most three and their elements project to zero in T. Hence even an arbitrarily long lower-degree sum *inside* $\mathbb{Q}(a)$ is impossible.

The Python verification independently enumerates all 60 subgroups of $S_3 \times S_3$ and reproduces these fixed-space rank calculations. The proof above does not require trusting that enumeration.

9 Arbitrarily many product factors: exact search and its limits

9.1 Two factors and many factors really differ

Consider

$$\eta = (1+\sqrt{2})(1+\sqrt{3})(1+\sqrt{5}).$$

This number has degree eight, as checked by its exact minimal polynomial in the verification script. Its displayed three-factor representation gives $D_{\times}(\eta) = 2$. A pair of quadratic factors cannot have degree-eight product. Nor can a pair with both degrees at most three: if one factor has degree three, the compositum has degree $3t$ with $1 \le t \le 3$, which is not divisible by eight; the remaining cases use only quadratic fields. Grouping two displayed factors into one degree-four factor gives

$$D_{\times,2}(\eta) = 4 > 2 = D_{\times}(\eta).$$

Thus a two-factor optimum cannot be promoted to a many-factor optimum. Repeatedly splitting an arbitrary chosen pair also provides no general proof that the final maximum degree is minimal; it explores only the factorization tree it happened to choose.

9.2 A finite dictionary with a precise meaning

For positive integers d, h, let $\mathcal{R}(d,h)$ be the roots of all primitive irreducible integer polynomials

$$c_0 + c_1 X + \cdots + c_m X^m, \qquad 1 \le m \le d, \quad |c_j| \le h, \quad 1 \le c_m \le h.$$

Each rational polynomial has a unique primitive integer normalization with positive leading coefficient, so every algebraic number of degree at most d occurs once its coefficient-height bound is sufficiently large. Nonmonic polynomials are necessary: $1/2$, for instance, has primitive minimal polynomial $2X - 1$.

The package enumerates coefficient vectors as base-$(2h+1)$ integers, rather than allocating all tuples in advance. It tests primitivity and `IrreduciblePolynomialQ`, enumerates every exact root, and removes canonical duplicates. The raw number of coefficient vectors grows as

$$\sum_{m=1}^{d} h(2h+1)^m.$$

This is intentionally a small-height discovery method, not an efficient replacement for subfield algorithms at large d.

9.3 Search the prefix and compute the last component exactly

`SearchRootDecomposition[a,op,d,h,r]` searches representations with at most r components. The first $r-1$ components, when needed, come from $\mathcal{R}(d,h)$; the last is an unrestricted exact residual, required only to have degree at most d. For a selected prefix, the residual is

$$a - \sum_{j<r} b_j \quad \text{or} \quad a \Big/ \prod_{j<r} b_j.$$

The dictionary therefore does not have to contain every component of a successful answer. Prefix indices are nondecreasing to avoid permutation duplicates; repetitions remain allowed. Product searches exclude zero.

At each node the exact residual degree is computed. A node is rejected if that degree exceeds $d^{\text{remaining slots}}$, or if its largest prime divisor exceeds d. Both tests are necessary conditions, so neither removes a valid

completion. As soon as the residual itself has degree at most d, it becomes the final component. A final `RootReduce` equality check and independent minimal-degree computations validate the candidate.

For the three-quadratic example, the first two factors have coefficient height two, so the following bounds suffice in the absence of resource termination:

```
eta = (1+Sqrt[2]) (1+Sqrt[3]) (1+Sqrt[5]);
SearchRootDecomposition[eta, "Product", 2, 2, 3]
(* a successful answer has maximum degree 2 and is globally optimal *)
```

The default node limit is 100,000, and the polynomial-enumeration limit is 100,000. These are operational guards, not mathematical height bounds.

Proposition 9.1 (What exhaustive bounded search establishes)**.** *With resource limits removed, the search is complete for representations having at most r components whose first $r-1$ components can be chosen in $\mathcal{R}(d,h)$. Letting h and r increase gives a positive-instance semidecision procedure for the degree cap d.*

Proof. Every eligible prefix occurs in the finite nondecreasing enumeration after reordering its commuting components. The residual test recognizes any valid last component, and the pruning conditions are necessary. For any particular finite representation at degree cap d, its finitely many primitive minimal polynomials have finite coefficient heights. Some pair of sufficiently large bounds therefore includes that representation. □

A result `NotFoundWithinBounds` means exactly that, not that d is impossible. Sequentially waiting for an exhaustive search at an infeasible degree cap can run forever. An anytime implementation can interleave the finite searches for different caps and height/length budgets, keep the best verified upper bound, and stop with a global certificate when it matches a valid lower bound. This article does not supply a proof that such a procedure terminates with a global certificate for every arbitrary-length product instance.

9.4 Why a many-factor norm argument needs more work

For $a = \prod_i b_i$, take a finite common extension M/L and set $t_i = [L(b_i) : L]$, $T = [M : L]$. Norms give

$$a^T = \prod_i \mathrm{N}_{L(b_i)/L}(b_i)^{T/t_i}. \tag{21}$$

The individual norm values lie in the intersections $L \cap \mathbb{Q}(b_i)$. Replacing each factor by a t_i-th root of its norm preserves an individual degree upper bound, but the product is only fixed *up to a root of unity*. Unlike the two-factor construction, one does not automatically obtain a common relative degree and an exact pairwise intersection equation.

Related radical replacement, with an explicit root-of-unity factor, is a central device in Samuels's metric Mahler-measure arguments [3, Theorem 2.1]. A theorem optimizing Mahler measure is not therefore a theorem optimizing absolute degree: roots of unity have Mahler measure one, while their degrees can be significant. For example,

$$\zeta_8 = \frac{1+i}{\sqrt{2}}$$

is a product of degree-two numbers although $\deg_{\mathbb{Q}}(\zeta_8) = 4$. Dropping a root-of-unity correction or inserting it as one supposedly low-degree component can invalidate a claimed degree bound. Equation (21) is useful structure, but not by itself a complete many-factor minimization algorithm.

10 Implementation details that affect correctness

10.1 Constructing exactly the intended Galois field

The backend begins by computing the rational minimal polynomial of the input and listing all its roots. It calls

```
common = ToNumberField[roots, All];
```

to put them in the *smallest* common field. The `All` argument is important: the documented automatic form need not choose the smallest common extension [7]. A larger normal field would still support the sum and pair criteria, but its group exponent need not give a valid lower bound for the original input. The package ties its Galois data to the exact input and uses the minimal splitting field.

A primitive generator is extracted from one of the resulting `AlgebraicNumber` entries. The generator is then multiplied by the leading coefficient of its primitive integer minimal polynomial, making it an algebraic integer without changing the field. This stabilizes `ToNumberField` coordinates: for an integral generator, the documented representation uses that generator [7]. The coordinates are read with `AlgebraicNumberPolynomial`, padded to the field dimension, and checked to be rational [8].

Every conjugate of the primitive generator lies in the splitting field. For each one, powers of its coordinate polynomial modulo the generator's minimal polynomial produce the corresponding automorphism matrix. The matrices must be distinct, contain the identity, and be closed under composition. These checks construct a full multiplication table for the Galois group without assuming a special built-in subfield or Galois-group API.

10.2 Enumerate embedded subgroups, not just conjugacy classes

Starting with the identity subgroup, the backend repeatedly adjoins an element and takes closure under the group multiplication table. Finite positive-power closure already supplies inverses. Duplicate subgroups are removed, and the process stops when no new subgroup appears.

One must keep all embedded subgroups. Replacing them by representatives up to conjugacy loses distinct subfields of the chosen copy of L and can miss a representation of the particular input a. Conjugacy compression is possible only if the missing embeddings are explicitly restored during the linear-algebra search.

For each subgroup H, the computed fixed-space dimension is checked against $|G|/|H|$. This is a useful independent invariant of the coordinate construction. A complete data association carries `CompleteSubfieldLattice -> True`; a resource-limited construction returns a failure instead of such an association.

10.3 Exact input, explicit exits, and certificate semantics

The package rejects decimal `Real` values. Rationalization of a numerical approximation is not an exact description of the intended algebraic number. It asks `MinimalPolynomial` for a rational polynomial and checks the coefficients before using its degree [6]. It does not inspect `First[a]` to guess a polynomial: an exact algebraic input may be a rational, a radical, a `Root`, or a compound expression.

Searches have nested loops and recursion. Wolfram's documented `Return` behavior can leave an inner loop rather than the intended outer routine [11]. The implementation therefore uses a unique `Catch`/`Throw` tag per routine invocation, including each recursive call. Search-wide resource exits have a separate tag. This keeps successful returns and failures from being accidentally swallowed by an inner control construct.

Every returned decomposition is rechecked against the immutable original input. The `Parts` list is the computational result; `Expression` contains an inactive sum or product for presentation. The flags distinguish exact verification, a two-factor optimum, a global optimum, and mere failure to find an answer within a budget. Equality and optimality are separate claims.

10.4 Public entry points

Function	Meaning and guarantee
`AlgebraicDegree`	Absolute minimal degree of an exact algebraic input.
`PrimeDegreeBound`	Lower bound valid for every finite sum/product representation.
`PolynomialRoots`	All exact branches of a nonconstant rational polynomial.
`SumPolynomial`, `ProductPolynomial`	Forward resultant constructions; outputs are annihilating polynomials, not necessarily minimal.
`CheckDecomposition`	Exact equality, component degrees, and a lightweight global certificate when the prime bound is attained.
`PairFromPolynomial`	Candidate-root enumeration plus an exact residual test. Failure is only for that polynomial.
`RootDictionary`	Finite, nonmonic-inclusive coefficient-height enumeration.
`SearchRootDecomposition`	Bounded many-component discovery; no negative global claim from a bounded miss.
`BuildGaloisData`	Explicit splitting field, automorphisms, subgroup lattice, and rational fixed-space bases.
`CompleteSumDecomposition`	Finite criterion for the global additive optimum, subject to successful backend construction.
`CompleteTwoFactorDecomposition`	Finite criterion for the global two-factor optimum, including external factors. Global many-factor optimality requires a matching lower bound.

10.5 Useful optimizations that do not change the mathematics

The deliberately direct backend can be improved without weakening its certificates. Cache field-coordinate vectors and multiplication matrices; store subgroup generators as well as elements; discard linearly redundant columns during the additive search; and test only degree caps at which the eligible fixed-field list changes. For pairs, cache the intersections and use E_0F_0 degree bounds to prune field pairs before constructing a large null-space matrix. Fraction-free or modular linear algebra can reduce coefficient growth, provided successful reconstructions are checked over $\mathbb{Q}$.

A more sophisticated number-field backend may compute subfields without materializing every automorphism matrix. It must still return all relevant embedded subfields with certified degrees and coordinates. Heuristic subfield discovery can accelerate positive cases, but cannot certify a negative degree cap unless completeness is separately established.

11 Reproducibility and verification status

The archive includes the article in source and compiled form, together with the Wolfram Language package, runnable examples, two native test suites, and independent verification code with its recorded output. The full package is also printed in Appendix A, so that the article exposes the implementation rather than hiding it behind function names.

Run the exact independent checks from the archive root with

```
python verification/verify.py
```

The recorded run used Python 3.13.5 and SymPy 1.14.0. It verifies the two resultants, irreducibility and real-root counts, rational and polynomial recovery certificates, the sextic pairing relation, and the degree-eight external-factor example. It also enumerates the order-36 and order-16 finite groups and checks their subgroup fixed-space obstructions exactly. All identity and degree acceptance tests use exact arithmetic. Numerical values are printed only for orientation.

The subgroup counts by index are

$G = S_3 \times S_3$, $[G : H]$	1	2	3	4	6	9	12	18	36
$\#H$	1	3	6	1	20	9	4	15	1

and

external example, $[G : H]$	1	2	4	8	16
$\#H$	1	7	7	7	1.

The finite-group calculations corroborate the proofs rather than replace them.

Native execution limitation. The available Wolfram evaluator returned HTTP 404, so it did not execute the package or tests. The Mathematica outputs in this article are explicitly expected results, not a claimed kernel transcript. The Python checks validate the algebra and group calculations, not Wolfram evaluation semantics or end-to-end performance. The supplied native suites separate basic functionality from expensive splitting-field tests:

```
TestReport["code/RootDecomposition.wlt"]
TestReport["code/ExtendedTests.wlt"]
```

The extended tests include the external quartic factors, the additive sextic optimum, and the distinction between two and three factors. Running these is the next implementation-validation step before using the package as a production simplifier.

To rebuild the article, run `latexmk -pdf article.tex`, or run `pdflatex article.tex` twice. The source uses ordinary TeX Live packages; no font binaries or external figures are required.

12 Conclusions

The supplied examples are not merely reducible to cubics: degree three is the provable global optimum for the indicated operation in each example. A small polynomial dictionary plus an exact residual calculation discovers the desired expressions without guessing both cubics simultaneously.

For arbitrary finite sums, trace projection turns the unrestricted problem in $\overline{\mathbb{Q}}$ into a finite rational fixed-space problem inside the splitting field. For two factors, norm descent gives a different finite criterion: enumerate a bounded radical exponent and test intersections $E_0 \cap a^t F_0$. This retains the auxiliary factors that a naive splitting-field-only search would discard.

The difference is substantive. The degree-eight number in Section 7 has $D_\times = 4$ but $D_+ = 8$; all lower-degree elements inside its splitting field lie in a proper subfield. The original degree-nine product has $D_\times = 3$ but $D_+ = 6$. These examples explain why the operation, allowed number of components, and ambient field must be explicit in both a mathematical theorem and a Mathematica API.

For products of an arbitrary number of factors, the delivered method is an exact bounded search with valid global certificates when available, not a claimed terminating universal minimizer. Keeping that distinction visible makes the successful answers meaningful: a displayed identity is verified, and a displayed optimum has an actual lower-bound proof.

References

[1] Vladimir Reshetnikov, *Decomposition of an algebraic number into a sum or product of algebraic numbers*, Mathematica StackExchange, question 105933, with subsequent answers. https://mathematica.stackexchange.com/q/105933/7288.

[2] Christian Altamirano, *A Problem in Algebraic Number Theory*, MIT UROP+ final paper, 31 August 2017. Mentor: Atticus Christensen; problem proposed by Bjorn Poonen. See the additive reduction and the multiplicative Theorem 3.1, discussed critically in Section 7. https://math.mit.edu/research/undergraduate/urop-plus/documents/2017/Altamirano.pdf.

[3] Charles L. Samuels, *The infimum in the metric Mahler measure*, arXiv:1408.4165, 2014. Especially Lemma 2.2 and Theorem 2.1. https://arxiv.org/abs/1408.4165.

[4] Wolfram Research, *Root*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Root.html.

[5] Wolfram Research, *RootReduce*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/RootReduce.html.

[6] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/MinimalPolynomial.html.

[7] Wolfram Research, *ToNumberField*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ToNumberField.html.

[8] Wolfram Research, *AlgebraicNumberPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html.

[9] Wolfram Research, *Resultant*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Resultant.html.

[10] Wolfram Research, *LinearSolve*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/LinearSolve.html.

[11] Wolfram Research, *Return*, Wolfram Language documentation, “Possible Issues” example on nested control constructs. https://reference.wolfram.com/language/ref/Return.html.

A Complete Wolfram Language reference implementation

This is the exact companion file `code/RootDecomposition.wl`. The mathematical completeness statements and the native-execution limitation are specified in the main text. The package depends on documented exact algebraic arithmetic and rational linear algebra, not on external number field software.

```
(* ::Package:: *)

(* Exact algebraic-number decompositions.
   See article.pdf for proofs, scope and verification status.
   All degree bounds are absolute degrees over Q. *)

BeginPackage["RootDecomposition`"];

RootDecomposition`AlgebraicDegree::usage = "AlgebraicDegree[a] is the degree over Q of an exact algebraic number.";
RootDecomposition`PrimeDegreeBound::usage = "PrimeDegreeBound[a] is a universal lower bound for the maximum degree in any finite sum or product representation.";
RootDecomposition`PolynomialRoots::usage = "PolynomialRoots[p,x] lists all exact roots of a nonconstant rational polynomial.";
RootDecomposition`SumPolynomial::usage = "SumPolynomial[p,q,x,z] gives the resultant for pairwise sums of roots.";
RootDecomposition`ProductPolynomial::usage = "ProductPolynomial[p,q,x,z] gives the resultant for pairwise products of roots.";
RootDecomposition`CheckDecomposition::usage = "CheckDecomposition[a,parts,op] verifies an exact sum/product and returns a certificate association. op is \"Sum\" or \"Product\".";
RootDecomposition`PairFromPolynomial::usage = "PairFromPolynomial[a,p,x,op,d] tries every root b of p and tests the degree of a-b or a/b.";
RootDecomposition`RootDictionary::usage = "RootDictionary[d,h] enumerates roots of primitive irreducible integer polynomials of degree at most d and coefficient height at most h.";
RootDecomposition`SearchRootDecomposition::usage = "SearchRootDecomposition[a,op,d,h,r] searches at most r parts; the first r-1 parts come from RootDictionary[d,h], while the final part is unrestricted apart from its degree.";
RootDecomposition`BuildGaloisData::usage = "BuildGaloisData[a] constructs the exact splitting field, automorphism matrices, all subgroups and fixed-field bases. This exhaustive backend is intended for small Galois groups.";
RootDecomposition`CompleteSumDecomposition::usage = "CompleteSumDecomposition[a,data] minimizes the maximum degree over all finite sums. data may be Automatic or BuildGaloisData[a].";
RootDecomposition`CompleteTwoFactorDecomposition::usage = "CompleteTwoFactorDecomposition[a,data] minimizes the maximum degree over two factors, including factors outside the splitting field. This is NOT a general many-factor minimizer.";

Begin["`Private`"];

(* Tagged exits are intentional. A bare Return inside Do can leave only
   the loop, not the enclosing routine. Each recursive call has its own tag. *)
SetAttributes[returnScope, HoldAll];
returnScope[body_] := Block[{$returnTag = Unique["return"]},
  Catch[body, $returnTag]];
returnValue[value_] := Throw[value, $returnTag];

rationalQ[x_] := IntegerQ[x] || Head[x] === Rational;
zeroQ[x_] := SameQ[Quiet[RootReduce[x]], 0];
validOpQ[op_] := MemberQ[{"Sum", "Product"}, op];
fail[tag_, text_] := Failure[tag, <|"MessageTemplate" -> text|>];

AlgebraicDegree[a_] := returnScope[Module[{x, p, v},
  If[!FreeQ[a, _Real],
    returnValue[fail["InexactInput", "Use an exact algebraic input, not a decimal approximation."]]];
  p = Quiet[Check[MinimalPolynomial[RootReduce[a], x], $Failed]];
  If[p === $Failed || !PolynomialQ[p, x],
    returnValue[fail["NotAlgebraic", "A rational minimal polynomial could not be computed."]]];
  v = CoefficientList[p, x];
  If[!VectorQ[v, rationalQ] || Exponent[p, x] < 1,
    returnValue[fail["NotAlgebraic", "A rational minimal polynomial could not be computed."]]];
  Exponent[p, x]
]];

primeBoundFromDegree[n_Integer] := If[n == 1, 1, Max[First /@ FactorInteger[n]]];
PrimeDegreeBound[a_] := returnScope[Module[{n = AlgebraicDegree[a]},
  If[FailureQ[n], n, primeBoundFromDegree[n]]
]];

PolynomialRoots[p_, x_Symbol] := returnScope[Module[{f, n},
  If[!PolynomialQ[p, x] || !VectorQ[CoefficientList[p, x], rationalQ],
```

```
    returnValue[fail["Polynomial", "Expected a polynomial with rational coefficients."]]];
  n = Exponent[p, x];
  If[n < 1, returnValue[fail["Polynomial", "Expected a nonconstant polynomial."]]];
  f = Function[Evaluate[p /. x -> Slot[1]]];
  Table[With[{j = k}, Root[f, j]], {k, n}]
]];

SumPolynomial[p_, q_, x_Symbol, z_Symbol] := returnScope[Module[{y},
  Expand[Resultant[p /. x -> y, q /. x -> z - y, y]]
]];
ProductPolynomial[p_, q_, x_Symbol, z_Symbol] := returnScope[Module[{y, m},
  m = Exponent[q, x];
  Expand[Resultant[p /. x -> y,
    Expand[Cancel[y^m (q /. x -> z/y)]], y]]
]];

CheckDecomposition[a_, parts_List, op_String] := returnScope[Module[
  {ps, ds, value, n, lb, expr},
  If[!validOpQ[op] || parts === {},
    returnValue[fail["Arguments", "Supply a nonempty list and operation Sum or Product."]]];
  ps = RootReduce /@ parts;
  ds = AlgebraicDegree /@ ps;
  n = AlgebraicDegree[a];
  If[FailureQ[n] || AnyTrue[ds, FailureQ],
    returnValue[fail["NotAlgebraic", "All entries must be exact algebraic numbers."]]];
  value = If[op == "Sum", Total[ps], Times @@ ps];
  If[!zeroQ[value - a],
    returnValue[fail["Equality", "The proposed decomposition failed exact verification."]]];
  lb = primeBoundFromDegree[n];
  expr = If[op == "Sum", Inactive[Plus] @@ ps, Inactive[Times] @@ ps];
  <|"Status" -> "Verified", "Input" -> RootReduce[a],
    "Operation" -> op, "Parts" -> ps, "Expression" -> expr,
    "Degrees" -> ds, "MaximumDegree" -> Max[ds], "InputDegree" -> n,
    "LowerBound" -> lb, "GlobalOptimal" -> (Max[ds] == lb),
    "OptimalityReason" -> If[Max[ds] == lb, "PrimeDivisorBound", "NotEstablished"]|>
]];

PairFromPolynomial[a_, p_, x_Symbol, op_String, d_Integer?Positive] := returnScope[Module[
  {rs, b, c, db, dc, result},
  If[!validOpQ[op], returnValue[fail["Operation", "Use Sum or Product."]]];
  rs = PolynomialRoots[p, x];
  If[FailureQ[rs], returnValue[rs]];
  Do[
    db = AlgebraicDegree[b];
    If[FailureQ[db] || db > d || (op == "Product" && zeroQ[b]), Continue[]];
    c = RootReduce[If[op == "Sum", a - b, a/b]];
    dc = AlgebraicDegree[c];
    If[IntegerQ[dc] && dc <= d,
      result = CheckDecomposition[a, {b, c}, op];
      If[!FailureQ[result], returnValue[result]]],
    {b, rs}];
  <|"Status" -> "NotFoundForThisPolynomial", "GlobalImpossibility" -> False|>
]];

Options[RootDictionary] = {"MaxPolynomials" -> 100000};
RootDictionary[d_Integer?Positive, h_Integer?Positive, OptionsPattern[]] := returnScope[Module[
  {x, m, lead, tail, coeff, p, roots = {}, count = 0,
   limit = OptionValue["MaxPolynomials"], cap, code, base},
  (* Streaming coefficient enumeration: do not allocate Tuples[-h..h,d]. *)
  base = 2 h + 1;
  Do[
    cap = base^m;
    Do[
      Do[
        count++;
        If[count > limit,
          returnValue[fail["ResourceLimit", "The polynomial-enumeration limit was reached."]]];
        tail = IntegerDigits[code, base, m] - h;
        coeff = Append[tail, lead];
        If[Apply[GCD, coeff] != 1, Continue[]];
        p = coeff . x^Range[0, m];
        If[TrueQ[IrreduciblePolynomialQ[p]],
          roots = Join[roots, PolynomialRoots[p, x]]],
```

```
        {code, 0, cap - 1}],
      {lead, 1, h}],
    {m, 1, d}];
  DeleteDuplicates[RootReduce /@ roots]
]];

Options[SearchRootDecomposition] = {
  "MaxNodes" -> 100000, "MaxPolynomials" -> 100000};
SearchRootDecomposition[a_, op_String, d_Integer?Positive,
    h_Integer?Positive, r_Integer?Positive, OptionsPattern[]] := returnScope[Module[
  {dict, walk, nodes = 0, limit = OptionValue["MaxNodes"], tag,
   parts, n, result},
  If[!validOpQ[op], returnValue[fail["Operation", "Use Sum or Product."]]];
  n = AlgebraicDegree[a];
  If[FailureQ[n], returnValue[n]];
  If[n <= d, returnValue[CheckDecomposition[a, {a}, op]]];
  If[primeBoundFromDegree[n] > d || n > d^r,
    returnValue[<|"Status" -> "ExcludedByDegreeBound", "DegreeCap" -> d,
      "MaximumParts" -> r,
      "GlobalImpossibilityAtDegreeCap" -> (primeBoundFromDegree[n] > d)|>]];
  dict = RootDictionary[d, h,
    "MaxPolynomials" -> OptionValue["MaxPolynomials"]];
  If[FailureQ[dict], returnValue[dict]];
  If[op == "Product", dict = Select[dict, !zeroQ[#] &]];
  tag = Unique["search"];
  walk[res_, slots_, start_, prefix_] := returnScope[Module[{nd, next, got, j},
    nodes++;
    If[nodes > limit, Throw[fail["ResourceLimit", "The search-node limit was reached."], tag]];
    nd = AlgebraicDegree[res];
    If[FailureQ[nd], Throw[nd, tag]];
    If[nd <= d, returnValue[Append[prefix, res]]];
    If[slots <= 1 || nd > d^slots || primeBoundFromDegree[nd] > d,
      returnValue[Missing["NotFound"]]];
    Do[
      next = RootReduce[If[op == "Sum", res - dict[[j]], res/dict[[j]]]];
      got = walk[next, slots - 1, j, Append[prefix, dict[[j]]]];
      If[ListQ[got], returnValue[got]],
      {j, start, Length[dict]}];
    Missing["NotFound"]
  ]];
  parts = Catch[walk[RootReduce[a], r, 1, {}], tag];
  If[FailureQ[parts], returnValue[parts]];
  If[!ListQ[parts], returnValue[<|"Status" -> "NotFoundWithinBounds",
    "DegreeCap" -> d, "HeightCap" -> h, "MaximumParts" -> r,
    "Nodes" -> nodes, "GlobalImpossibility" -> False|>]];
  result = CheckDecomposition[a, parts, op];
  If[FailureQ[result], result, Join[result, <|"Nodes" -> nodes|>]]
]];

(* The small-field exact backend. Vectors use 1,theta,...,theta^(N-1).
   Fixed-field bases are stored as ROW vectors. Automorphisms and
   multiplication operators act on COLUMN coordinate vectors. *)

integerPolynomial[a_, x_] := returnScope[Module[{p, c, den, gcd},
  p = MinimalPolynomial[a, x]; c = CoefficientList[p, x];
  den = Apply[LCM, Denominator /@ c]; c = den c;
  gcd = Apply[GCD, c];
  Expand[(Sign[Last[c]]/gcd) den p]
]];

makeField[theta0_] := returnScope[Module[{x, p, theta, n},
  p = integerPolynomial[theta0, x];
  theta = RootReduce[Coefficient[p, x, Exponent[p, x]] theta0];
  p = integerPolynomial[theta, x]; n = Exponent[p, x];
  <|"Generator" -> theta, "Polynomial" -> p,
    "Variable" -> x, "Dimension" -> n|>
]];

fieldVector[field_, a_] := returnScope[Module[{an, p, c, x = field["Variable"], n},
  n = field["Dimension"];
  an = Quiet[Check[ToNumberField[RootReduce[a], field["Generator"]], $Failed]];
  If[an === $Failed || !(rationalQ[an] || Head[an] === AlgebraicNumber),
    returnValue[fail["FieldMembership", "Could not express an element in the chosen field."]]];
```

```
  If[Head[an] === AlgebraicNumber && !zeroQ[an[[1]] - field["Generator"]],
    returnValue[fail["GeneratorChanged", "The coordinate generator changed unexpectedly."]]];
  p = AlgebraicNumberPolynomial[an, x];
  c = CoefficientList[p, x];
  If[!VectorQ[c, rationalQ] || Length[c] > n,
    returnValue[fail["Coordinates", "Expected rational power-basis coordinates."]]];
  PadRight[c, n]
]];

fieldElement[field_, v_List] := RootReduce[
  v . field["Generator"]^Range[0, field["Dimension"] - 1]];

polyVector[field_, p_] := PadRight[
  CoefficientList[p, field["Variable"]], field["Dimension"]];

powerMatrix[field_, cv_] := returnScope[Module[
  {x = field["Variable"], p = field["Polynomial"], n = field["Dimension"],
   c, w = 1, rows = {}, j},
  c = cv . x^Range[0, n - 1];
  Do[
    AppendTo[rows, polyVector[field, w]];
    w = PolynomialRemainder[Expand[w c], p, x], {j, n}];
  Transpose[rows]
]];

multiplicationMatrix[field_, av_] := returnScope[Module[
  {x = field["Variable"], p = field["Polynomial"], n = field["Dimension"], f},
  f = av . x^Range[0, n - 1];
  Transpose[Table[polyVector[field,
    PolynomialRemainder[Expand[f x^j], p, x]], {j, 0, n - 1}]]
]];

groupClosure[gens_List, table_, id_Integer] := returnScope[Module[
  {elts = {id}, cursor = 1, x, y, g},
  While[cursor <= Length[elts],
    x = elts[[cursor]]; cursor++;
    Do[y = table[[x, g]];
      If[!MemberQ[elts, y], AppendTo[elts, y]], {g, gens}]];
  Sort[elts]
]];

allSubgroups[table_, id_Integer, limit_] := returnScope[Module[
  {groups = {{id}}, cursor = 1, h, k, g, m = Length[table]},
  While[cursor <= Length[groups],
    h = groups[[cursor]]; cursor++;
    Do[
      If[MemberQ[h, g], Continue[]];
      k = groupClosure[Append[h, g], table, id];
      If[!MemberQ[groups, k],
        AppendTo[groups, k];
        If[Length[groups] > limit,
          returnValue[fail["ResourceLimit", "The subgroup-enumeration limit was reached."]]]],
      {g, m}]];
  groups
]];

groupExponent[table_, id_] := returnScope[Module[{orders, x, k, i},
  orders = Table[x = i; k = 1;
    While[x != id, x = table[[x, i]]; k++]; k,
    {i, Length[table]}];
  Apply[LCM, orders]
]];

Options[BuildGaloisData] = {"MaxFieldDegree" -> 128, "MaxSubgroups" -> 10000};
BuildGaloisData[a_, OptionsPattern[]] := returnScope[Module[
  {n, x, roots, common, entries, theta, field, m, conjugates, cvs, mats,
   idpos, id, table, pos, groups, subfields, h, basis, exponent},
  n = AlgebraicDegree[a]; If[FailureQ[n], returnValue[n]];
  If[n == 1, theta = 1,
    roots = PolynomialRoots[MinimalPolynomial[a, x], x];
    (* All is important: Automatic may introduce an unnecessarily large field. *)
    common = Quiet[Check[ToNumberField[roots, All], $Failed]];
    If[common === $Failed,
```

```
      returnValue[fail["PrimitiveElement", "Could not construct the splitting field."]]];
    entries = Cases[common, _AlgebraicNumber, {1}];
    If[entries === {}, returnValue[fail["PrimitiveElement", "No primitive generator was returned."]]];
    theta = entries[[1, 1]]];
  field = makeField[theta]; m = field["Dimension"];
  If[m > OptionValue["MaxFieldDegree"],
    returnValue[fail["ResourceLimit", "The splitting field exceeds MaxFieldDegree."]]];
  conjugates = PolynomialRoots[field["Polynomial"], field["Variable"]];
  cvs = fieldVector[field, #] & /@ conjugates;
  If[AnyTrue[cvs, FailureQ],
    returnValue[fail["NotGalois", "Not all conjugates could be embedded in the constructed field."]]];
  mats = powerMatrix[field, #] & /@ cvs;
  If[Length[DeleteDuplicates[mats]] != m,
    returnValue[fail["Automorphisms", "The automorphism matrices were not distinct."]]];
  idpos = FirstPosition[mats, IdentityMatrix[m]];
  If[MissingQ[idpos], returnValue[fail["Automorphisms", "Identity automorphism missing."]]];
  id = First[idpos];
  table = Table[
    pos = FirstPosition[cvs, mats[[i]] . cvs[[j]]];
    If[MissingQ[pos], 0, First[pos]], {i, m}, {j, m}];
  If[MemberQ[table, 0, Infinity],
    returnValue[fail["Automorphisms", "Automorphisms did not form a closed group."]]];
  groups = allSubgroups[table, id, OptionValue["MaxSubgroups"]];
  If[FailureQ[groups], returnValue[groups]];
  subfields = Table[
    basis = NullSpace[Join @@ ((# - IdentityMatrix[m]) & /@ mats[[h]])];
    If[Length[basis] != m/Length[h],
      returnValue[fail["FixedField", "A fixed-field dimension failed the index check."]]];
    <|"Subgroup" -> h, "Degree" -> Length[basis], "BasisRows" -> basis|>,
    {h, groups}];
  exponent = groupExponent[table, id];
  <|"Input" -> RootReduce[a], "InputDegree" -> n, "Field" -> field,
    "GroupOrder" -> m, "GroupExponent" -> exponent,
    "Automorphisms" -> mats, "GroupTable" -> table, "Identity" -> id,
    "Subfields" -> SortBy[subfields, #["Degree"] &],
    "CompleteSubfieldLattice" -> True|>
]];

galoisBound[data_] := returnScope[Module[{n = data["InputDegree"], e = data["GroupExponent"], b},
  b = SelectFirst[Range[n], Mod[Apply[LCM, Range[#]], e] == 0 &];
  Max[primeBoundFromDegree[n], b]
]];

resolveData[a_, data_] := returnScope[Module[{g},
  g = If[data === Automatic, BuildGaloisData[a], data];
  If[FailureQ[g], returnValue[g]];
  If[!AssociationQ[g] || !TrueQ[Lookup[g, "CompleteSubfieldLattice", False]] ||
      !zeroQ[g["Input"] - a],
    returnValue[fail["Data", "Use the complete BuildGaloisData result for this exact input."]]];
  g
]];

CompleteSumDecomposition[a_, data_: Automatic] := returnScope[Module[
  {g, field, n, low, target, eligible, rows, mat, coefficients,
   parts, offset, b, k, result, d},
  n = AlgebraicDegree[a]; If[FailureQ[n], returnValue[n]];
  If[n == 1, returnValue[CheckDecomposition[a, {a}, "Sum"]]];
  g = resolveData[a, data]; If[FailureQ[g], returnValue[g]];
  field = g["Field"]; low = galoisBound[g]; target = fieldVector[field, a];
  If[FailureQ[target], returnValue[target]];
  Do[
    eligible = Select[g["Subfields"], #["Degree"] <= d &];
    rows = Join @@ (#["BasisRows"] & /@ eligible);
    mat = Transpose[rows];
    coefficients = Quiet[Check[LinearSolve[mat, target], $Failed]];
    If[!ListQ[coefficients] || !VectorQ[coefficients, rationalQ], Continue[]];
    If[mat . coefficients =!= target, Continue[]];
    parts = {}; offset = 0;
    Do[
      b = entry["BasisRows"]; k = Length[b];
      AppendTo[parts, fieldElement[field,
        Take[coefficients, {offset + 1, offset + k}] . b]];
      offset += k,
```

```
      {entry, eligible}];
    parts = Select[parts, !zeroQ[#] &];
    If[parts === {}, parts = {0}];
    result = CheckDecomposition[a, parts, "Sum"];
    If[FailureQ[result], returnValue[result]];
    returnValue[Join[result, <|"LowerBound" -> d, "GlobalOptimal" -> True,
      "OptimalityReason" -> "CompleteGaloisFixedSpaceSearch",
      "GaloisGroupOrder" -> g["GroupOrder"]|>]],
    {d, low, n}];
  fail["InternalCheck", "The full field should contain a valid sum decomposition."]
]];

CompleteTwoFactorDecomposition[a_, data_: Automatic] := returnScope[Module[
  {g, field, n, low, eligible, av, mult, e, f, mat, ns, c, uv, u,
   beta, gamma, result, i, j, d, t},
  n = AlgebraicDegree[a]; If[FailureQ[n], returnValue[n]];
  If[n == 1,
    result = CheckDecomposition[a, {a, 1}, "Product"];
    returnValue[Join[result, <|"OptimalAmongTwoFactors" -> True|>]]];
  g = resolveData[a, data]; If[FailureQ[g], returnValue[g]];
  field = g["Field"]; low = Max[galoisBound[g], Ceiling[Sqrt[n]]];
  Do[
    Do[
      eligible = Select[g["Subfields"], t #["Degree"] <= d &];
      av = fieldVector[field, RootReduce[a^t]];
      If[FailureQ[av], returnValue[av]];
      mult = multiplicationMatrix[field, av];
      Do[
        e = eligible[[i]]["BasisRows"];
        Do[
          f = eligible[[j]]["BasisRows"];
          mat = Join[Transpose[e], -mult . Transpose[f], 2];
          ns = NullSpace[mat]; If[ns === {}, Continue[]];
          c = First[ns]; uv = Take[c, Length[e]] . e;
          u = fieldElement[field, uv];
          If[zeroQ[u], returnValue[fail["InternalCheck", "An intersection basis gave zero."]]];
          beta = RootReduce[u^(1/t)]; gamma = RootReduce[a/beta];
          result = CheckDecomposition[a, {beta, gamma}, "Product"];
          If[FailureQ[result], returnValue[result]];
          If[result["MaximumDegree"] > d,
            returnValue[fail["InternalCheck", "The norm-descent degree bound failed."]]];
          returnValue[Join[result, <|"OptimalAmongTwoFactors" -> True,
            "TwoFactorLowerBound" -> d, "NormExponent" -> t,
            "LowerBound" -> galoisBound[g],
            "GlobalOptimal" -> (result["MaximumDegree"] == galoisBound[g]),
            "OptimalityReason" -> If[result["MaximumDegree"] == galoisBound[g],
              "GaloisExponentAndDegreeBound", "TwoFactorsOnly"],
            "GaloisGroupOrder" -> g["GroupOrder"]|>]],
          {j, i, Length[eligible]}],
        {i, Length[eligible]}],
      {t, 1, Min[d, Floor[d^2/n]]}],
    {d, low, n}];
  fail["InternalCheck", "The pair {a,1} should always be found."]
]];

End[];
EndPackage[];
```